Modelado de datos con estructuras (struct) y punteros

Malek Tomas Cheheid Tubello

EEST N.º 1 “Eduardo Ader”, Vicente López

Laboratorio de Programación (LPR)

5.° año, 3.° grupo A

Docente: Yamil Ganduglia

15 de octubre de 2026

Modalidad: Individual

Repositorio: https://github.com/malekcheheid-art/sistemaproyecto

Modelado de datos con estructuras (struct) y punteros

Introducción

Este informe documenta la Actividad 5 de la materia Laboratorio de Programación (LPR) de 5.° año, 3.° grupo A. El trabajo modela una entidad real mediante una estructura de datos (struct) en C++ y controla el ingreso y la modificación de sus miembros por referencia, a través de punteros y del operador flecha (->).

La actividad forma parte del Proyecto Integrador Anual. La estructura que se define aquí es la base para organizar las variables principales del prototipo electrónico presentado en la feria tecnológica (Ganduglia, 2026).

Objetivos

Los objetivos del trabajo son los siguientes:

- Modelar una entidad del proyecto mediante una estructura (struct) en C++.
- Controlar el ingreso y la modificación de sus miembros mediante punteros.
- Aplicar el operador flecha (->) y el pasaje por referencia.
- Explicar el funcionamiento del buffer de entrada y de la instrucción cin.ignore().

Marco teórico

Estructuras (struct)

Una struct es una colección de variables de distintos tipos agrupadas bajo una misma entidad lógica. En la memoria RAM sus miembros se almacenan en bloques contiguos, uno detrás del otro. Esto permite tratar un conjunto de datos relacionados como una sola unidad, en lugar de declarar variables sueltas por separado (cplusplus.com, s.f.).

Puntero y operador flecha (->)

Un puntero es una variable que almacena la dirección de memoria de otra variable. El operador flecha (->) es la sintaxis que usa C++ para acceder a los miembros de una struct a través de un puntero: primero desreferencia el puntero y luego accede al campo. Su

equivalente es la expresión (*puntero).miembro (cplusplus.com, s.f.).

Pasaje por referencia

Consiste en enviar la dirección de memoria (&) de un objeto a una función. De este modo, la función trabaja sobre el objeto original y no sobre una copia: los cambios se reflejan en la variable que se pasó. Como los datos no se duplican, también se reduce el uso de memoria en la pila (stack).

Buffer de entrada y cin.ignore()

El buffer de entrada es el espacio de memoria donde el sistema guarda los caracteres que se ingresan por teclado antes de que el programa los lea. La instrucción cin.ignore() descarta el carácter Enter (\n) que queda remanente, de manera que la lectura posterior con cin.getline no se omite.

Desarrollo práctico

El programa se encuentra en el archivo src/main.cpp. Define la struct EntidadProyecto, que representa un componente del prototipo: un identificador numérico, un nombre o descripción y una métrica de operación. El seguimiento del desarrollo se documentó en el repositorio de GitHub con las ramas main y development (Chacón & Straub, 2014).

Definición de la estructura

```
struct EntidadProyecto {
    int   id;
    char  nombre[50];
    float metrica;
};
```

La struct reúne los tres datos que describen a un sensor del sistema. Los miembros se almacenan juntos en memoria y el conjunto completo ocupa 60 bytes (sizeof(EntidadProyecto) = 60, verificados al compilar). Ese dato es importante para entender el pasaje por referencia que se explica más adelante.

Ingreso de datos por referencia

```
void cargarDatos(EntidadProyecto* ptr) {
    cin >> ptr->id;
    cin.ignore();
    cin.getline(ptr->nombre, 50);
    cin >> ptr->metrica;
}
```

La función recibe un puntero a la estructura y escribe los datos ingresados por el usuario con el operador flecha. Cada operación ptr->campo desreferencia el puntero y luego accede al miembro. En la llamada se envía la dirección del objeto con el operador (&miEntidad).

Lectura y verificación de los datos

La función mostrarDatos() recibe el mismo puntero declarado como const, es decir, en modo solo lectura: puede leer los miembros pero no modificarlos. Imprime los valores cargados y la dirección de memoria real de la estructura, lo que confirma que el puntero opera sobre la variable original y no sobre una copia. El código completo del programa se incluye en el Apéndice B.

Resultados

El programa se compiló con g++ (MSYS2) en Windows y se ejecutó desde la terminal PowerShell. Como datos de prueba se ingresaron los siguientes valores: identificador 7, descripción “Sensor de temperatura - aula 3B” y métrica 27.5. La salida mostró los tres valores cargados y la dirección de memoria de la estructura en formato hexadecimal (0xe9837ff900 en esta ejecución). Esa dirección corresponde a la misma variable declarada en main(), lo que confirma que las funciones trabajaron sobre el objeto original. La evidencia de la ejecución se encuentra en la carpeta capturas/ y se incluye como Figura 1 en el Apéndice A.

Cuestionario técnico

1. Heterogeneidad

Una struct reúne datos de distintos tipos bajo un mismo nombre: en este caso, un entero, una cadena de caracteres y un valor decimal. Frente a declarar variables sueltas, agruparlos en una estructura tiene tres ventajas: los datos relacionados quedan bajo una misma entidad lógica; se almacenan en memoria de forma contigua; y se pasan a una función como un solo argumento en lugar de una lista larga de parámetros, lo que reduce el riesgo de mezclar datos de entidades distintas.

2. Operador flecha

La notación de punto (ptr.id) no se puede usar cuando ptr es un puntero, porque un puntero no es una estructura: solo contiene una dirección de memoria. C++ no aplica la desreferencia en forma automática porque sería ambigua con la aritmética de punteros. Por eso existe el operador flecha (->), que desreferencia el puntero y accede al miembro: ptr->id es equivalente a (*ptr).id.

3. Eficiencia en RAM

El pasaje por valor copia toda la estructura en la pila en cada llamada a la función, con el costo de memoria y de tiempo que eso implica. El pasaje por referencia transfiere únicamente la dirección: 8 bytes en un sistema de 64 bits. La función lee o escribe directamente en el objeto original. En el caso de EntidadProyecto, cuyo tamaño es de 60 bytes, cada llamada evita una copia completa de la estructura.

4. Buffer de entrada

Cuando se lee con cin >>, la tecla Enter no se consume: el carácter \n queda almacenado en el buffer. Si a continuación se ejecuta cin.getline, esa función lee ese \n como primer carácter y termina de inmediato. El resultado es que el texto escrito por el usuario se omite y hay que presionar Enter otra vez, con la impresión de que el programa se saltó un paso. La instrucción cin.ignore() descarta el carácter sobrante del buffer y evita el problema.

5. Aplicación al proyecto integrador

La estructura modela un componente de hardware del sistema: un sensor de temperatura instalado en el aula. Sus miembros son el identificador del sensor (id), la descripción del componente (nombre) y la lectura de temperatura en grados centígrados (metrica). El mismo modelo se puede reutilizar para los demás sensores del proyecto, agregando un campo por cada magnitud que mida cada uno.

Conclusiones

Las estructuras permitieron agrupar, bajo una misma entidad, datos de distintos tipos que pertenecen al mismo componente. El pasaje por referencia con punteros posibilitó modificar el objeto original sin generar copias en memoria, y el operador flecha (->) dio una sintaxis clara para acceder a sus miembros. También se comprobó por qué es necesario limpiar el buffer de entrada antes de leer texto con cin.getline. Estos conceptos son la base para organizar las variables principales del Proyecto Integrador Anual.

Referencias

cplusplus.com. (s. f.). *Estructuras de datos en C++ (struct)*. https://cplusplus.com/doc/tutorial/structures/

cplusplus.com. (s. f.). *Punteros en C++*. https://cplusplus.com/doc/tutorial/pointers/

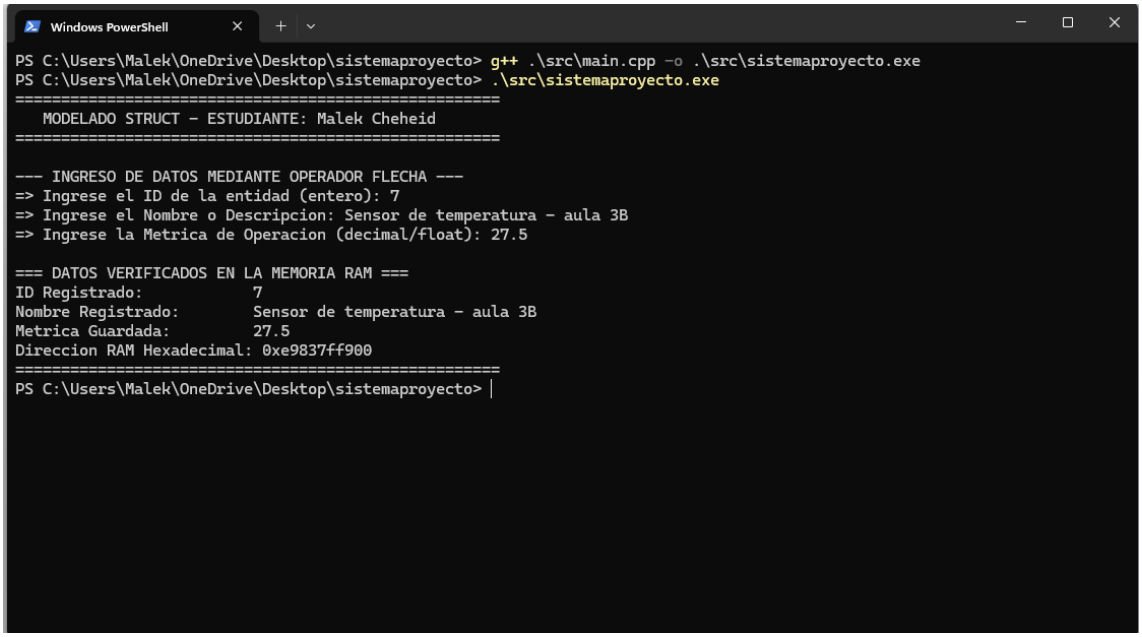
Chacón, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Ganduglia, Y. (2026). *Actividad 5: Modelado de datos con estructuras (struct) y punteros* [Consigna de trabajo]. Laboratorio de Programación, EEST N.º 1 “Eduardo Ader”.

Apéndice A

Figura 1

Captura de la ejecución del programa



Apéndice B

Código completo (src/main.cpp)

```
/*
================================================================
Materia: Laboratorio de Programacion (LPR) - 5to ano
Establecimiento: EEST N.o 1 "Eduardo Ader" - Vicente Lopez
Alumno: MALEK TOMAS CHEHEID TUBELLO - Grupo A

Archivo: src/main.cpp

Actividad 5: Modelado de datos con estructuras (struct) y punteros.
Muestra el pasaje por referencia mediante punteros y el uso del
operador flecha (->) para modificar los miembros de una estructura.
================================================================
*/

#include <iostream>
#include <cstring>

using namespace std;

/*
1. DEFINICION DE LA ESTRUCTURA
Agrupa en un unico bloque contiguo de memoria los datos de una
entidad del proyecto integrador anual.
*/
struct EntidadProyecto {
    int   id;          // Identificador numerico del componente
    char  nombre[50];  // Nombre o descripcion del componente
    float metrica;     // Lecturas de sensores, consumo, avance, etc.
};

/* 2. PROTOTIPO
   Recibe la direccion de memoria de la estructura (puntero). */
void cargarDatos(EntidadProyecto* ptr);

/* Muestra los datos leyendo la estructura con notacion de punto. */
void mostrarDatos(const EntidadProyecto* ptr);

int main() {

    /* Inicializacion de seguridad en la memoria Stack */
    EntidadProyecto miEntidad = {0, "Vacio - Malek Cheheid", 0.0f};

    cout << "=====================================================" << endl;
    cout << "   MODELADO STRUCT - ESTUDIANTE: Malek Cheheid      " << endl;
    cout << "=====================================================" << endl;

    /* 3. INVOCACION
       Se envia la direccion fisica de memoria con '&', de modo que
       la funcion modifique el original y no una copia. */
    cargarDatos(&miEntidad);

    cout << "\n=== DATOS VERIFICADOS EN LA MEMORIA RAM ===" << endl;

    mostrarDatos(&miEntidad);
```

```
    cout << "=====================================================" << endl;

    return 0;  // Codigo 0: finalizacion exitosa
}

/*
4. Ingreso de datos por referencia.
   El puntero 'ptr' accede a los campos con el operador flecha (->),
   equivalente a desreferenciar el puntero y luego usar el punto.
*/
void cargarDatos(EntidadProyecto* ptr) {

    cout << "\n--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---" << endl;

    cout << "=> Ingrese el ID de la entidad (entero): ";
    cin >> ptr->id;

    /* Limpieza del buffer obligatoria antes de cin.getline,
       para eliminar el Enter remanente. */
    cin.ignore();

    cout << "=> Ingrese el Nombre o Descripcion: ";
    cin.getline(ptr->nombre, 50);

    cout << "=> Ingrese la Metrica de Operacion (decimal/float): ";
    cin >> ptr->metrica;

}

/* 5. Verificacion: los datos leidos confirman la modificacion
      del objeto original, no de una copia. */
void mostrarDatos(const EntidadProyecto* ptr) {

    cout << "ID Registrado:            " << ptr->id << endl;
    cout << "Nombre Registrado:        " << ptr->nombre << endl;
    cout << "Metrica Guardada:         " << ptr->metrica << endl;

    /* Con el punto se accede a la estructura en memoria. */
    cout << "Direccion RAM Hexadecimal: " << &(*ptr) << endl;

}
```